

SKILL OSSP WEEK 6 (S 11 & 12)

SESSION 11:

TASK 1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define HISTORY_SIZE 5
#define INPUT_SIZE 1024

typedef struct
{
    char commands[HISTORY_SIZE][INPUT_SIZE];
    int count;
    int next_position;
} History;

void initialize_history(History *history)
{
    history->count = 0;
    history->next_position = 0;
}

void add_command(History *history, const char *command)
{
    if (command == NULL || strlen(command) == 0)
    {
        return;
    }

    strncpy(
        history->commands[history->next_position],
        command,
        INPUT_SIZE - 1
    );

    history->commands[history->next_position][INPUT_SIZE - 1] = '\0';

    history->next_position =
        (history->next_position + 1) % HISTORY_SIZE;

    if (history->count < HISTORY_SIZE)
    {
        history->count++;
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2
printf("\nCommand History:\n");

if (history->count == 0)
{
    printf("History is empty.\n");
    return;
}

int start;

if (history->count < HISTORY_SIZE)
{
    start = 0;
}
else
{
    start = history->next_position;
}

for (int i = 0; i < history->count; i++)
{
    int index = (start + i) % HISTORY_SIZE;
    printf("%d: %s\n", i + 1, history->commands[index]);
}

const char *get_history_entry(
    const History *history,
    int entry_number
)
{
    if (entry_number < 1 || entry_number > history->count)
    {
        return NULL;
    }

    int start;

    if (history->count < HISTORY_SIZE)
    {
        start = 0;
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
| if (history->count < HISTORY_SIZE)
| {
|     start = 0;
| }
| else
| {
|     start = history->next_position;
| }
|
| int index = (start + entry_number - 1) % HISTORY_SIZE;
| return history->commands[index];
| }
|
| int validate_history(const History *history)
| {
|     if (history->count < 0 || history->count > HISTORY_SIZE)
|     {
|         return 0;
|     }
|
|     if (history->next_position < 0 ||
|         history->next_position >= HISTORY_SIZE)
|     {
|         return 0;
|     }
|
|     return 1;
| }
|
| int main()
| {
|     History history;
|     char input[INPUT_SIZE];
|
|     initialize_history(&history);
|
|     printf("Command History Manager - Session 11 Task 1\n");
|     printf("History capacity: %d commands\n", HISTORY_SIZE);
|     printf("Commands: history, get N, check, exit\n\n");
|
|     while (1)
|     {
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
|
|     while (1)
|     {
|         printf("history-shell> ");
|         fflush(stdout);
|
|         if (fgets(input, sizeof(input), stdin) == NULL)
|         {
|             break;
|         }
|
|         input[strcspn(input, "\n")] = '\0';
|
|         if (strcmp(input, "exit") == 0)
|         {
|             break;
|         }
|
|         if (strcmp(input, "history") == 0)
|         {
|             display_history(&history);
|             continue;
|         }
|
|         if (strcmp(input, "check") == 0)
|         {
|             if (validate_history(&history))
|             {
|                 printf("History consistency: VALID\n");
|             }
|             else
|             {
|                 printf("History consistency: INVALID\n");
|             }
|             continue;
|         }
|
|         if (strncmp(input, "get ", 4) == 0)
|         {
|             int number = atoi(input + 4);
|         }
|     }
| }
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
    else
    {
        printf("History consistency: INVALID\n");
    }
    continue;
}

if (strncmp(input, "get ", 4) == 0)
{
    int number = atoi(input + 4);

    const char *entry =
        get_history_entry(&history, number);

    if (entry != NULL)
    {
        printf("History entry %d: %s\n",
            number,
            entry);
    }
    else
    {
        printf("Invalid history entry.\n");
    }
    continue;
}

add_command(&history, input);

printf("Command stored successfully.\n");
}

printf("\nFinal history:\n");
display_history(&history);

printf("History manager terminated.\n");

return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
koumudi@Koumudi:~$ cd ~/OSSP/SKILL
koumudi@Koumudi:~/OSSP/SKILL$ ls
Session_1 Session_10 Session_2 Session_3 Session_4 Session_5 Session_6 Session_7 Session_8 Session_9
koumudi@Koumudi:~/OSSP/SKILL$ mkdir Session_11
koumudi@Koumudi:~/OSSP/SKILL$ cd Session_11
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ pwd
/home/koumudi/OSSP/SKILL/Session_11
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ nano history_shell.c
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ gcc history_shell.c -o history_shell
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ ./history_shell
Command History Manager - Session 11 Task 1
History capacity: 5 commands
Commands: history, get N, check, exit

history-shell> ls
Command stored successfully.
history-shell> pwd
Command stored successfully.
history-shell> date
Command stored successfully.
history-shell> whoami
Command stored successfully.
history-shell> echo Hello
Command stored successfully.
history-shell> history

Command History:
1: ls
2: pwd
3: date
4: whoami
5: echo Hello
history-shell> ls -l
Command stored successfully.
history-shell> get 1
History entry 1: pwd
history-shell> get 5
History entry 5: ls -l
history-shell> check
History consistency: VALID
```

```
history-shell> check
History consistency: VALID
history-shell> exit

Final history:

Command History:
1: pwd
2: date
3: whoami
4: echo Hello
5: ls -l
History manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ |
```

TASK 2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

#define INPUT_SIZE 1024
#define MAX_COMMANDS 5
#define MAX_ARGS 64

typedef struct
{
    char commands[MAX_COMMANDS][INPUT_SIZE];
    int count;
} Pipeline;

/* Remove leading and trailing spaces */
char *trim(char *str)
{
    while (isspace((unsigned char)*str))
    {
        str++;
    }

    if (*str == '\0')
    {
        return str;
    }

    char *end = str + strlen(str) - 1;

    while (end > str && isspace((unsigned char)*end))
    {
        end--;
    }

    *(end + 1) = '\0';

    return str;
}

/* Parse commands separated by | */
int parse_pipeline(char *input, Pipeline *pipeline)
{
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
{
    pipeline->count = 0;
    char *command = strtok(input, "|");
    while (command != NULL)
    {
        command = trim(command);
        if (strlen(command) == 0)
        {
            return 0;
        }
        if (pipeline->count >= MAX_COMMANDS)
        {
            printf("Error: Maximum %d commands allowed.\n",
                MAX_COMMANDS);
            return 0;
        }
        strncpy(
            pipeline->commands[pipeline->count],
            command,
            INPUT_SIZE - 1
        );
        pipeline->commands[pipeline->count][INPUT_SIZE - 1] = '\0';
        pipeline->count++;
        command = strtok(NULL, "|");
    }
    return pipeline->count > 0;
}

/* Validate pipeline structure */
int validate_pipeline(const Pipeline *pipeline)
{
    if (pipeline->count < 1 ||
        pipeline->count > MAX_COMMANDS)
    {
        return 0;
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
    }
    return 0;
}

for (int i = 0; i < pipeline->count; i++)
{
    if (strlen(pipeline->commands[i]) == 0)
    {
        return 0;
    }
}

return 1;
}

/* Display pipeline layout */
void display_pipeline(const Pipeline *pipeline)
{
    printf("\nPipeline Layout:\n");

    for (int i = 0; i < pipeline->count; i++)
    {
        printf("Stage %d: %s\n",
            i + 1,
            pipeline->commands[i]);

        if (i < pipeline->count - 1)
        {
            printf("    |\n");
            printf("    v\n");
        }
    }

    printf("\nExecution order: ");

    for (int i = 0; i < pipeline->count; i++)
    {
        printf("%d", i + 1);

        if (i < pipeline->count - 1)
        {
            printf(" -> ");
        }
    }

    printf("\n");
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
/* Convert one command into arguments */
int tokenize_command(char *command, char **args)
{
    int count = 0;

    char *token = strtok(command, " \\t");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " \\t");
    }

    args[count] = NULL;

    return count;
}

/* Execute all commands connected by pipes */
int execute_pipeline(Pipeline *pipeline)
{
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    char command_copies[MAX_COMMANDS][INPUT_SIZE];
    char *args[MAX_COMMANDS][MAX_ARGS];

    /* Create copies because tokenization modifies strings */
    for (int i = 0; i < pipeline->count; i++)
    {
        strncpy(
            command_copies[i],
            pipeline->commands[i],
            INPUT_SIZE - 1
        );

        command_copies[i][INPUT_SIZE - 1] = '\\0';

        if (tokenize_command(command_copies[i], args[i]) == 0)
        {
            printf("Error: Empty command at stage %d.\\n", i + 1);
            return 0;
        }
    }
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
History manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ nano pipeline_shell.c
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ gcc pipeline_shell.c -o pipeline_shell
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ l
L: command not found
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ ls
history_shell history_shell.c pipeline_shell pipeline_shell.c
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ ./pipeline_shell
Pipeline Manager - Session 11 Task 2
Maximum pipeline stages: 5
Use | to connect commands.
Type 'exit' to quit.

pipeline-shell> ls | wc -l

Pipeline validation: VALID

Pipeline Layout:
Stage 1: ls
|
v
Stage 2: wc -l

Execution order: 1 -> 2

Executing pipeline...
4
Pipeline execution completed.

pipeline-shell> echo hello world | tr a-z A-Z | wc -c

Pipeline validation: VALID

Pipeline Layout:
Stage 1: echo hello world
|
v
Stage 2: tr a-z A-Z
|
v
Stage 3: wc -c

Execution order: 1 -> 2 -> 3

Executing pipeline...
12
```

```
koumudi@Koumudi: ~/OSSP/ × + v

Executing pipeline...
12
Pipeline execution completed.

pipeline-shell> printf "apple\nbanana\napple\n" | grep apple | wc -l

Pipeline validation: VALID

Pipeline Layout:
Stage 1: printf "apple\nbanana\napple\n"
    |
    v
Stage 2: grep apple
    |
    v
Stage 3: wc -l

Execution order: 1 -> 2 -> 3

Executing pipeline...
2
Pipeline execution completed.

pipeline-shell> ls | | wc -l
Invalid pipeline layout.
pipeline-shell> echo hello | tr a-z A-Z | cat | cat | wc -c

Pipeline validation: VALID

Pipeline Layout:
Stage 1: echo hello
    |
    v
Stage 2: tr a-z A-Z
    |
```

```
Pipeline Layout:
Stage 1: echo hello
    |
    v
Stage 2: tr a-z A-Z
    |
    v
Stage 3: cat
    |
    v
Stage 4: cat
    |
    v
Stage 5: wc -c

Execution order: 1 -> 2 -> 3 -> 4 -> 5

Executing pipeline...
6
Pipeline execution completed.

pipeline-shell> exit
Pipeline manager terminated.
koumudi@Koumudi:~/OSSP/SKILL/Session_11$ |
```

SESSION 12:

TASK 1: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

```
koumudi@Koumudi: ~/OSSP/ x + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define INPUT_SIZE 1024

int main()
{
    int pipe_fd[2];

    pid_t child1;
    pid_t child2;

    char message[INPUT_SIZE];

    printf("Pipe Data Flow - Session 12 Task 1\n");
    printf("-----\n");

    /* Create pipe */
    if (pipe(pipe_fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");
    printf("Read descriptor: %d\n", pipe_fd[0]);
    printf("Write descriptor: %d\n", pipe_fd[1]);

    /* Create first child */
    child1 = fork();

    if (child1 == -1)
    {
```

```
GNU nano 7.2
{
    perror("fork");
    return 1;
}

if (child1 == 0)
{
    /* First child writes data into pipe */

    close(pipe_fd[0]);

    const char *data =
        "Hello from Process 1 through the pipe!";

    write(
        pipe_fd[1],
        data,
        strlen(data) + 1
    );

    printf("Process 1: Data written to pipe.\n");

    close(pipe_fd[1]);

    exit(0);
}

/* Create second child */
child2 = fork();

if (child2 == -1)
{
    perror("fork");
    return 1;
}
```

```
koumudi@Koumudi: ~/OSSP/ x + v
GNU nano 7.2

if (child2 == 0)
{
    /* Second child reads data from pipe */

    close(pipe_fd[1]);

    ssize_t bytes_read =
        read(
            pipe_fd[0],
            message,
            sizeof(message)
        );

    if (bytes_read > 0)
    {
        printf("Process 2: Data received from pipe:\n");
        printf("%s\n", message);
    }
    else
    {
        perror("read");
    }

    close(pipe_fd[0]);

    exit(0);
}

/* Parent closes its copies of descriptors */
close(pipe_fd[0]);
close(pipe_fd[1]);

printf("Parent: Pipe descriptors closed.\n");

/* Synchronize with both child processes */
waitpid(child1, NULL, 0);
waitpid(child2, NULL, 0);

printf("Parent: Both processes completed.\n");
printf("Data flow test completed successfully.\n");

return 0;
}
#include <stdio.h>
```

```
koumudi@Koumudi: ~/OSSP/ x + v
GNU nano 7.2
}
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define INPUT_SIZE 1024

int main()
{
    int pipe_fd[2];

    pid_t child1;
    pid_t child2;

    char message[INPUT_SIZE];

    printf("Pipe Data Flow - Session 12 Task 1\n");
    printf("-----\n");

    /* Create pipe */
    if (pipe(pipe_fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");
    printf("Read descriptor: %d\n", pipe_fd[0]);
    printf("Write descriptor: %d\n", pipe_fd[1]);

    /* Create first child */
    child1 = fork();

    if (child1 == -1)
    {
        perror("fork");
        return 1;
    }

    if (child1 == 0)
    {
        /* First child writes data into pipe */
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
{
    /* First child writes data into pipe */
    close(pipe_fd[0]);

    const char *data =
        "Hello from Process 1 through the pipe!";

    write(
        pipe_fd[1],
        data,
        strlen(data) + 1
    );

    printf("Process 1: Data written to pipe.\n");

    close(pipe_fd[1]);

    exit(0);
}

/* Create second child */
child2 = fork();

if (child2 == -1)
{
    perror("fork");
    return 1;
}

if (child2 == 0)
{
    /* Second child reads data from pipe */
    close(pipe_fd[1]);

    ssize_t bytes_read =
        read(
            pipe_fd[0],
            message,
            sizeof(message)
        );

    if (bytes_read > 0)
```

```
    {
        if (bytes_read > 0)
        {
            printf("Process 2: Data received from pipe:\n");
            printf("%s\n", message);
        }
        else
        {
            perror("read");
        }

        close(pipe_fd[0]);

        exit(0);
    }

    /* Parent closes its copies of descriptors */
    close(pipe_fd[0]);
    close(pipe_fd[1]);

    printf("Parent: Pipe descriptors closed.\n");

    /* Synchronize with both child processes */
    waitpid(child1, NULL, 0);
    waitpid(child2, NULL, 0);

    printf("Parent: Both processes completed.\n");
    printf("Data flow test completed successfully.\n");

    return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾

koumudi@Koumudi:~/OSSP/SKILL$ mkdir Session_12
koumudi@Koumudi:~/OSSP/SKILL$ cd Session_12
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ pwd
/home/koumudi/OSSP/SKILL/Session_12
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ nano pipe_dataflow.c
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ ls
pipe_dataflow.c
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ gcc pipe_dataflow.c -o pipe_dataflow
pipe_dataflow.c:123:5: error: redefinition of 'main'
   123 | int main()
       |     ^~~~
pipe_dataflow.c:9:5: note: previous definition of 'main' with type 'int()'
     9 | int main()
       |     ^~~~
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ gcc pipe_dataflow.c -o pipe_dataflow
pipe_dataflow.c:123:5: error: redefinition of 'main'
   123 | int main()
       |     ^~~~
pipe_dataflow.c:9:5: note: previous definition of 'main' with type 'int()'
     9 | int main()
       |     ^~~~
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ nano pipe_dataflow.c
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ ls
pipe_dataflow.c
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ gcc pipe_dataflow.c -o pipe_dataflow
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ ./pipe_dataflow
Session 12 Task 1 - Pipe Data Flow
Creating pipe successfully...
Parent: Pipe descriptors closed.
Producer: Sending data...
Producer: Data sent and write descriptor closed.
Consumer: Received data: Hello from producer process!
Consumer: Read descriptor closed.
Parent: Producer and consumer processes synchronized.
Data flow test completed successfully.
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ ls -l
total 24
-rwxr-xr-x 1 koumudi koumudi 16440 Sep 18 08:23 pipe_dataflow
-rw-r--r-- 1 koumudi koumudi 1834 Sep 18 08:23 pipe_dataflow.c
koumudi@Koumudi:~/OSSP/SKILL/Session_12$
```

TASK 2: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define INPUT_SIZE 1024
#define MAX_COMMANDS 6
#define MAX_ARGS 64

typedef struct
{
    char commands[MAX_COMMANDS][INPUT_SIZE];
    int count;
} Pipeline;

/* Remove leading and trailing spaces */
char *trim(char *str)
{
    while (*str == ' ' || *str == '\t')
    {
        str++;
    }

    if (*str == '\0')
    {
        return str;
    }

    char *end = str + strlen(str) - 1;

    while (end > str &&
           (*end == ' ' || *end == '\t'))
    {
        end--;
    }

    *(end + 1) = '\0';

    return str;
}

/* Parse commands separated by | */
int parse_pipeline(char *input, Pipeline *pipeline)
{
    pipeline->count = 0;

    char *command = strtok(input, "|");

    while (command != NULL)
    {

```

```
GNU nano 7.2
    {
        command = trim(command);

        if (strlen(command) == 0)
        {
            return 0;
        }

        if (pipeline->count >= MAX_COMMANDS)
        {
            printf("Error: Maximum %d commands allowed.\n",
                   MAX_COMMANDS);

            return 0;
        }

        strncpy(
            pipeline->commands[pipeline->count],
            command,
            INPUT_SIZE - 1
        );

        pipeline->commands[pipeline->count]
            [INPUT_SIZE - 1] = '\0';

        pipeline->count++;

        command = strtok(NULL, "|");
    }

    return pipeline->count > 0;
}

/* Validate pipeline */
int validate_pipeline(const Pipeline *pipeline)
{
    if (pipeline->count < 2 ||
        pipeline->count > MAX_COMMANDS)
    {
        return 0;
    }

    for (int i = 0; i < pipeline->count; i++)
    {
        if (strlen(pipeline->commands[i]) == 0)
        {
            return 0;
        }
    }

    return 1;
}
```



koumudi@Koumudi: ~/OSSP/



GNU nano 7.2

```
/* Display pipeline information */
void display_pipeline(const Pipeline *pipeline)
{
    printf("\nPipeline Configuration:\n");

    printf("Number of commands: %d\n",
           pipeline->count);

    for (int i = 0; i < pipeline->count; i++)
    {
        printf("Process %d: %s\n",
               i + 1,
               pipeline->commands[i]);

        if (i < pipeline->count - 1)
        {
            printf("      |\n");
            printf("      v\n");
        }
    }

    printf("\nExecution order: ");

    for (int i = 0; i < pipeline->count; i++)
    {
        printf("P%d", i + 1);

        if (i < pipeline->count - 1)
        {
            printf(" -> ");
        }
    }

    printf("\n");
}

/* Convert command into argv format */
int tokenize_command(char *command, char **args)
{
    int count = 0;

    char *token = strtok(command, " \t");

    while (token != NULL &&
           count < MAX_ARGS - 1)
    {
        args[count++] = token;

        token = strtok(NULL, " \t");
    }
}
```



koumudi@Koumudi: ~/OSSP/



GNU nano 7.2

```
    args[count] = NULL;
    return count;
}

/* Execute a pipeline containing multiple processes */
int execute_pipeline(Pipeline *pipeline)
{
    int pipe_fds[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];
    char command_copies[MAX_COMMANDS][INPUT_SIZE];
    char *args[MAX_COMMANDS][MAX_ARGS];

    /*
     * Make separate copies of commands
     * because strtok modifies the strings.
     */
    for (int i = 0; i < pipeline->count; i++)
    {
        strncpy(
            command_copies[i],
            pipeline->commands[i],
            INPUT_SIZE - 1
        );

        command_copies[i][INPUT_SIZE - 1] = '\0';

        if (tokenize_command(
            command_copies[i],
            args[i]) == 0)
        {
            printf("Error: Empty command at process %d.\n",
                i + 1);
            return 0;
        }
    }

    /*
     * Create multiple pipes.
     *
     * For N processes, N-1 pipes are required.
     */
    for (int i = 0;
         i < pipeline->count - 1;
         i++)
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
printf("Invalid pipeline configuration.\n\n");
    continue;
}

/*
 * Validate the pipeline.
 */
if (!validate_pipeline(
    &pipeline))
{
    printf("Pipeline validation: INVALID\n\n");
    continue;
}

printf("\nPipeline validation: VALID\n");

/*
 * Display pipeline configuration.
 */
display_pipeline(&pipeline);

printf("\nStarting pipeline execution...\n");

/*
 * Execute pipeline.
 */
if (execute_pipeline(&pipeline))
{
    printf("All pipeline processes completed.\n");
}
else
{
    printf("Pipeline execution failed.\n");
}

printf("\n");

printf("Multiple pipe manager terminated.\n");

return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ ls
pipe_dataflow  pipe_dataflow.c
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ nano multiple_pipes.c
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ gcc multiple_pipes.c -o multiple_pipes
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ ls
multiple_pipes  multiple_pipes.c  pipe_dataflow  pipe_dataflow.c
koumudi@Koumudi:~/OSSP/SKILL/Session_12$ ./multiple_pipes
Multiple Pipe Manager - Session 12 Task 2
Maximum pipeline processes: 6
For N processes, N-1 pipes are created.
Type 'exit' to quit.

multi-pipe> echo hello world | tr a-z A-Z | wc -c

Pipeline validation: VALID

Pipeline Configuration:
Number of commands: 3
Process 1: echo hello world
|
v
Process 2: tr a-z A-Z
|
v
Process 3: wc -c

Execution order: P1 -> P2 -> P3

Starting pipeline execution...
Created pipe 1: read=3 write=4
Created pipe 2: read=5 write=6
Process 1 completed.
Process 2 completed.
12
Process 3 completed.
All pipeline processes completed.

multi-pipe> printf "apple\nbanana\napple\norange\napple\n" | grep apple | sort | uniq | wc -l

Pipeline validation: VALID

Pipeline Configuration:
Number of commands: 5
Process 1: printf "apple\nbanana\napple\norange\napple\n"
```

```

koumudi@Koumudi: ~/OSSP/ X + v
Number of commands: 5
Process 1: printf "apple\nbanana\napple\norange\napple\n"
|
v
Process 2: grep apple
|
v
Process 3: sort
|
v
Process 4: uniq
|
v
Process 5: wc -l

Execution order: P1 -> P2 -> P3 -> P4 -> P5

Starting pipeline execution...
Created pipe 1: read=3 write=4
Created pipe 2: read=5 write=6
Created pipe 3: read=7 write=8
Created pipe 4: read=9 write=10
Process 1 completed.
Process 2 completed.
Process 3 completed.
Process 4 completed.
2
Process 5 completed.
All pipeline processes completed.

multi-pipe> ls | grep ".c" | sort | head -5 | wc -l

Pipeline validation: VALID

Pipeline Configuration:
Number of commands: 5
Process 1: ls
|
v
Process 2: grep ".c"
|
v
Process 3: sort
|
v
Process 4: head -5

```

```

koumudi@Koumudi: ~/OSSP/ X + v
Process 4: head -5
|
v
Process 5: wc -l

Execution order: P1 -> P2 -> P3 -> P4 -> P5

Starting pipeline execution...
Created pipe 1: read=3 write=4
Created pipe 2: read=5 write=6
Created pipe 3: read=7 write=8
Created pipe 4: read=9 write=10
Process 1 completed.
Process 2 completed.
0
Process 3 completed.
Process 4 completed.
Process 5 completed.
All pipeline processes completed.

multi-pipe> echo hello | | wc -l
Invalid pipeline configuration.

multi-pipe> echo hello | tr a-z A-Z | cat | cat | cat | wc -c

Pipeline validation: VALID

Pipeline Configuration:
Number of commands: 6
Process 1: echo hello
|
v
Process 2: tr a-z A-Z
|
v
Process 3: cat
|
v
Process 4: cat
|
v
Process 5: cat
|
v
Process 6: wc -c

```

```

v
Process 6: wc -c

Execution order: P1 -> P2 -> P3 -> P4 -> P5 -> P6

Starting pipeline execution...
Created pipe 1: read=3 write=4
Created pipe 2: read=5 write=6
Created pipe 3: read=7 write=8
Created pipe 4: read=9 write=10
Created pipe 5: read=11 write=12
Process 1 completed.
Process 2 completed.
Process 3 completed.
Process 4 completed.
Process 5 completed.
6
Process 6 completed.
All pipeline processes completed.

multi-pipe> exit
Multiple pipe manager terminated.
koumudi@Koumudi: ~/OSSP/SKILL/Session_12$

```